

PYTHON - SE - BACKEND

SOFTWARE ENGINEERING – MODULE 4

Database Management – SQL & PL/SQL

SESSION / ASSIGNMENT – 9

TASKS :

1. Create a SQL script to insert a new order into an Orders table, then use COMMIT to save the transaction and verify that the new order persists after reconnecting to the database.

➔ **CODE:** Inserting a new order into Orders table and COMMIT to save transaction

START TRANSACTION;

```
INSERT INTO Orders (order_id, user_id, order_date, order_total)
VALUES (101, 1, NOW(), 1250.00);
```

COMMIT;

Verifying the new order

```
SELECT *
FROM Orders
WHERE order_id = 101;
```

OUTPUT:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```
START TRANSACTION;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row inserted. (Query took 0.0007 seconds.)

```
INSERT INTO Orders (order_id, user_id, order_date, order_total) VALUES (101, 1, NOW(), 1250.00);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0019 seconds.)

```
COMMIT;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

2. **Simulate a Zomato-style food order process: insert two new items into an OrderItems table, then use ROLLBACK to undo the changes before committing. Check that no new items remain in the table after rollback.**

➔ CODE: Creating Table OrderItems

```
CREATE TABLE OrderItems (  
    item_id INT PRIMARY KEY,  
    order_id INT,  
    product_name VARCHAR(100),  
    quantity INT,  
    price DECIMAL(10,2)  
) ENGINE = InnoDB;
```

Performing Transaction

```
START TRANSACTION;
```

```
INSERT INTO OrderItems  
(item_id, order_id, product_name, quantity, price)  
VALUES  
(201, 101, 'Pizza', 1, 500.00);
```

```
INSERT INTO OrderItems
(item_id, order_id, product_name, quantity, price)
VALUES
(202, 101, 'Burger', 2, 300.00);
```

```
ROLLBACK;
```

Checking no new items remain in the table after rollback

```
SELECT *
FROM OrderItems
WHERE item_id IN (201, 202);
```

OUTPUT:

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
START TRANSACTION;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row inserted. (Query took 0.0003 seconds.)

```
INSERT INTO OrderItems (item_id, order_id, product_name, quantity, price) VALUES (201, 101, 'Pizza', 1, 500.00);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ 1 row inserted. (Query took 0.0021 seconds.)

```
INSERT INTO OrderItems (item_id, order_id, product_name, quantity, price) VALUES (202, 101, 'Burger', 2, 300.00);
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0008 seconds.)

```
ROLLBACK;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
SELECT * FROM OrderItems WHERE item_id IN (201, 202);
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

item_id	order_id	product_name	quantity	price
---------	----------	--------------	----------	-------

Query results operations

3. In a Flipkart-like shopping cart scenario, use SAVEPOINT to mark a point after adding a product to the Cart table, add another product, then use ROLLBACK TO SAVEPOINT to undo only the last addition. Show the final contents of the Cart table.

➔ **CODE:** use SAVEPOINT to mark a point

```
START TRANSACTION;
```

Adding first product

```
INSERT INTO Cart
(cart_id, user_id, product_id, product_name, quantity)
VALUES
(301, 1, 501, 'Laptop', 1);
```

Created savepoint

```
SAVEPOINT product_added;
```

Adding second product

```
INSERT INTO Cart
(cart_id, user_id, product_id, product_name, quantity)
VALUES
(302, 1, 502, 'Wireless Mouse', 1);
```

Undo only the second product

```
ROLLBACK TO SAVEPOINT product_added;
```

Show final contents

```
SELECT *
FROM Cart
WHERE user_id = 1;
```

OUTPUT:

	cart_id	user_id	product_id	product_name	quantity
☐ Edit Copy Delete	301	1	501	Laptop	1
☐ Check all With selected: Edit Copy Delete Export					

4. Write a trigger on a Wallet table that automatically deducts the purchase amount from the user's balance whenever a new transaction is inserted, similar to how Paytm updates wallet balance after a payment.

➔ **CODE:**

```
DELIMITER //
```

```
CREATE TRIGGER deduct_wallet_balance
AFTER INSERT ON WalletTransactions
FOR EACH ROW
BEGIN
    IF NEW.transaction_type = 'PURCHASE' THEN
        UPDATE Wallet
        SET balance = balance - NEW.amount
        WHERE user_id = NEW.user_id;
    END IF;
END //
```

```
DELIMITER ;
```

Testing the trigger

```
SELECT *
FROM Wallet
WHERE user_id = 1;
```

Taking 750 from the Wallet

```
INSERT INTO WalletTransactions
(transaction_id, user_id, amount, transaction_type)
VALUES
(401, 1, 750.00, 'PURCHASE');
```

Checking the wallet

```
SELECT *
FROM Wallet
WHERE user_id = 1;
```

OUTPUT:

	user_id	balance
	1	5000.00

	user_id	balance
	1	4250.00

5. Use ChatGPT to generate a SQL test scenario where a trigger incorrectly updates a user's balance after an expense entry. Paste the scenario and your corrected trigger code, explaining how you fixed the error.

Hint: Ask ChatGPT for a buggy trigger example and a test case that exposes the bug.

→ Buggy trigger

Suppose we accidentally write:

```
DELIMITER //
```

```
CREATE TRIGGER buggy_expense_trigger
AFTER INSERT ON WalletTransactions
FOR EACH ROW
BEGIN
    IF NEW.transaction_type = 'EXPENSE' THEN
        UPDATE Wallet
        SET balance = balance + NEW.amount
        WHERE user_id = NEW.user_id;
    END IF;
END //
```

```
DELIMITER ;
```

What is the mistake?

The trigger uses:

```
balance = balance + NEW.amount
```

But an **expense should decrease the wallet balance**, not increase it.

Test scenario

Suppose “user 1” has:

Initial balance = ₹5000

Expense = ₹1000

Insert the expense:

```
INSERT INTO WalletTransactions  
(transaction_id, user_id, amount, transaction_type)  
VALUES  
(402, 1, 1000.00, 'EXPENSE');
```

The buggy trigger calculates:

$5000 + 1000 = ₹6000$

But the correct calculation should be:

$5000 - 1000 = ₹4000$

Therefore, the test exposes the bug.

Corrected trigger

First remove the buggy trigger:

```
DROP TRIGGER IF EXISTS buggy_expense_trigger;
```

Now create the corrected version:

```
DELIMITER //
```

```
CREATE TRIGGER correct_expense_trigger  
AFTER INSERT ON WalletTransactions  
FOR EACH ROW  
BEGIN  
    IF NEW.transaction_type = 'EXPENSE' THEN  
        UPDATE Wallet  
        SET balance = balance - NEW.amount  
        WHERE user_id = NEW.user_id;  
    END IF;  
END //
```

DELIMITER ;

Now test it:

```
INSERT INTO WalletTransactions  
(transaction_id, user_id, amount, transaction_type)  
VALUES  
(403, 1, 1000.00, 'EXPENSE');
```

Check the balance:

```
SELECT *  
FROM Wallet  
WHERE user_id = 1;
```

		user_id	balance
	 Edit  Copy  Delete	1	4250.00